

Objects, Files, and ArrayLists

Overview and objectives

- **Course:** CIS 3300, Intermediate Object-Oriented Programming (Java)
- **Texts:** *Core Java, Volume I* (13th ed.) and *Core Java, Volume II* (13th ed.)
- **This week:** one in-class demo (*Objects, Files, and ArrayLists*) and one individual lab (*Kanto League Roster Lab*)
- **Prerequisite you already have:** basic `ArrayList` operations (`add`, `get`, `size`) and one-dimensional arrays

1. The short version

This week moves in two directions at once. First, a light introduction to objects: enough to create one from a class someone else wrote and call its methods, not yet to design a class from scratch. Second, three ways to get data into a Java program (a space-separated text file, a CSV file read by hand, and a CSV file read with a library) and one way to get it back out. The `ArrayList` work from before comes back too, now used to keep a list in ranked order as it grows, and to find and update an object already in a list rather than only adding new ones.

Class time is spent on the demo, five short examples that each show one technique on its own. The lab is where those techniques combine into a single program.

2. What We Will Cover

- **Objects, briefly.** Creating an object from a provided class with its constructor, and reading and changing its fields with getters and setters. Full class design comes later.
- **Reading a space-separated file.** Using `Scanner`'s `next()`, `nextInt()`, and `nextDouble()`, which split on whitespace automatically.
- **Reading a CSV file by hand.** Splitting each line with `split(",")`, and where that quietly falls apart.
- **Reading a CSV file with OpenCSV.** What a real CSV library gets right that manual splitting does not.
- **Writing to a file.** `PrintWriter`, and why closing the file when you are done matters.
- **`ArrayList` as more than a container.** Inserting an object into its ranked position as you read, and searching an existing list to update an object already there.

3. Objectives: By the End, You Should Be Able To

1. **Create an object** from a class someone else wrote, using its constructor, and call its getters and setters.
2. **Read a space-separated text file** with `Scanner`, using `next()`, `nextInt()`, and `nextDouble()` to read one value at a time.
3. **Read a CSV file by hand** with `split(",")`, and explain why that approach can silently produce wrong data on messier input.
4. **Read a CSV file with the OpenCSV library**, and explain what problem it solves that `split(",")` does not.
5. **Write formatted output to a file** with `PrintWriter`, and explain why the file has to be closed when you are done writing to it.
6. **Insert an object into an ArrayList** at a specific ranked position as you read data, rather than adding everything to the end and sorting afterward.
7. **Search an ArrayList** for an object that already exists in it, and update that object instead of adding a duplicate.
8. **Combine two files** describing the same data, matched by a shared ID, into a single `ArrayList` of objects.

4. How to Show Up Ready

Before lab, read the chapters from *Core Java, Volume I* and *Volume II* assigned on the course schedule for this week. Also read through the *Objects, Files, and ArrayLists* demo handout, and make sure each of its five parts makes sense on its own: building an object by hand, reading a space-separated file, reading a CSV file two different ways, and writing a file back out. Come to the *Kanto League Roster Lab* ready to combine two files into one ranked list. That combining step is the one genuinely new idea this week; everything else is a technique from the demo, applied to a new pair of files.